

3) LINEAR METHODS FOR REGRESSION

Assume we have a dataset with 3 predictors (x_1, x_2, x_3) and 1 dependent variable y :

We write the input vector as:

	x_1	x_2	x_3	y
1 sample	1	2	1	3
1 predictor	3	1	5	2
	7	1	2	3

$$X = \begin{bmatrix} 1 & 2 & 1 \\ 3 & 1 & 5 \\ 7 & 1 & 2 \end{bmatrix} \text{ Matrix Form}$$

Similarly, the output vector is $Y = \begin{bmatrix} 3 \\ 2 \\ 3 \end{bmatrix}$

If we have p predictors and n data samples, then, X has dimension $n \times p$ (n rows and p columns). Y has dimension $n \times 1$.

3.2) LINEAR REGRESSION AND LEAST SQUARES

The linear regression model, has form

$$Y = X\beta + E \iff \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}_{n \times 1} = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}_{n \times k} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_k \end{bmatrix}_{k \times 1} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \vdots \\ \epsilon_n \end{bmatrix}_{n \times 1}$$

For a single observation, this becomes:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_k x_k + E$$

The linear regression model has high bias as it assumes that $E[Y|X]$ is linear (or assuming linearity is reasonable).

To estimate $\beta = [\beta_0, \beta_1, \dots, \beta_p]^T$ we use a set of training data with N observations $(x_1, y), \dots, (x_N, y_N)$:

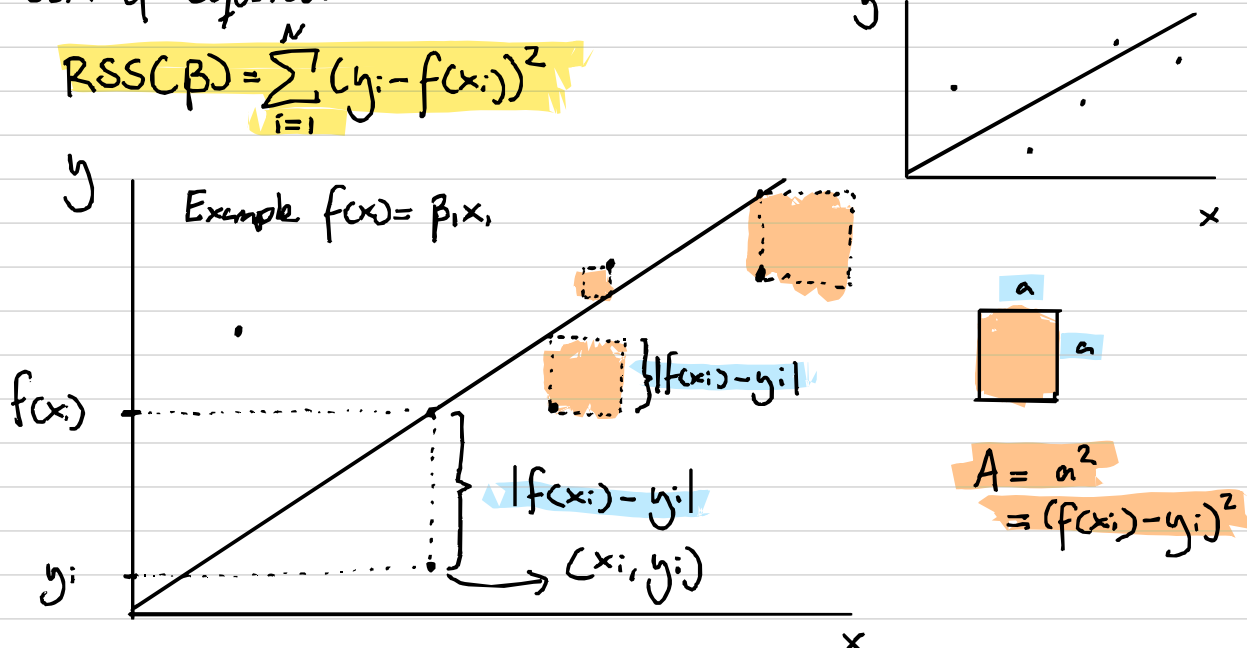
$$- x_i \in \mathbb{R}^p, \quad x_i = [x_{i1}, x_{i2}, \dots, x_{ip}]$$

$$- y_i \in \mathbb{R}$$

Let f be our linear model: $f: \mathbb{R}^p \rightarrow \mathbb{R}$
 $x: t \mapsto y:$

Then, the least squares method minimises the residual sum of squares:

$$RSS(\beta) = \sum_{i=1}^N (y_i - f(x_i))^2$$



In matrix form:

$$RSS(\beta) = (Y - X\beta)^T (Y - X\beta)$$

The minimum is obtained by setting:

$$\frac{\partial RSS}{\partial \beta} = 0 \implies -2X^T Y + 2X^T X \beta = 0$$

$$\implies X^T X \beta = X^T Y$$

$$\text{OLS Solution} \implies \hat{\beta} = (X^T X)^{-1} X^T Y$$

*) Assuming that X has full column rank, hence $X^T X$ is invertible and positive definite.

$$\hat{Y} = X \hat{\beta} \quad * \hat{\cdot} \text{ denotes an estimate}$$

$$= X (X^T X)^{-1} X^T Y \quad (\text{From OLS})$$

$$= H Y \quad (\text{Letting } H = X (X^T X)^{-1} X^T)$$

H is sometimes called the hat matrix. Do you see why?

If the columns of X are not linearly independent (e.g. $x_1 = 2x_2$), then β is not uniquely defined. In this case, the OLS formula does not hold. This also happens if $p > N$.

The variance of the estimator is:

$$\text{Var}(\hat{\beta}) = (X^T X)^{-1} \sigma^2, \quad \hat{\sigma}^2 = \frac{1}{N-p-1} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$\hat{\sigma}^2$ is an unbiased estimate of σ^2 . If we assume the conditional expectation of Y ($E[Y|X]$) is linear on $X_1, \dots, X_p$, and Gaussian errors, then.

$$Y = E(Y|X_1, \dots, X_p) + E$$

$$= \beta_0 + \sum_{j=1}^p X_j \beta_j + E \quad E \sim N(0, \sigma^2)$$

Then, $\hat{\beta} \sim N(\beta, (X^T X)^{-1} \sigma^2) \implies$ This can be used to construct hypothesis tests and confidence intervals for β_j

The test hypothesis for a particular coefficient $\beta_j = 0$.

$$\text{Z-score: } z_j = \frac{\hat{\beta}_j}{\hat{\sigma} \sqrt{v_j}} \quad v_j: j\text{-th diagonal element of } (X^T X)^{-1}$$

z_j is distributed as t_{N-p-1} . If σ is known (not estimated), then z_j is normally distributed. For $N-p-1 > 100$ $t_{N-p-1} \sim N$.

F-statistic

$$F = \frac{\frac{RSS_0 - RSS_1}{p_1 - p_0}}{\frac{RSS_1}{n - p_1}}$$

RSS_1 : bigger model
 RSS_0 : smaller model
 p_1, p_0 : n° of predictor (β_0 counts)
 N : n° of training examples

Example

Baseline (model 0): $F(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_4 x_4$

- $RSS_0 = 32.81$ (Residual sum of squares on data)

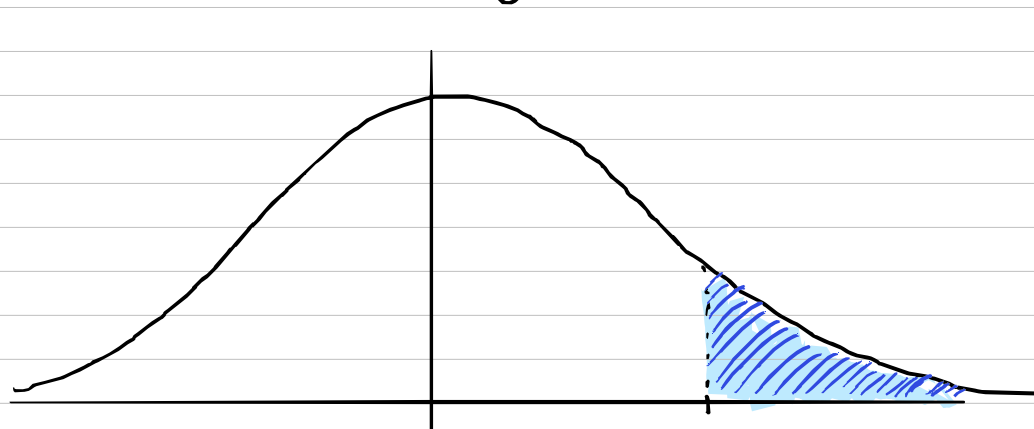
New (model 1): $F(x) = \beta_0 + \beta_1 x_1 + \dots + \beta_8 x_8$ (add 4 predictors)

- $RSS_1 = 29.43$

If both models are trained on 67 data examples, then:

$$F = \frac{(32.81 - 29.43) / (9 - 5)}{29.43 / (67 - 9)} = 1.67$$

$$P(F_{4,58} > 1.67) = 0.17 \implies \text{Not significant}$$



Exercises (Certificate of engagement)

1) Create a linear regression class in Python which:

- Creates a train/test split
- Computes linear regression via OLS
- Computes the residual sum of squares of the model
- Computes the hat matrix
- Computes the variance of the estimate $\hat{\beta}$
- Computes the z-scores of $\hat{\beta}$

2) Create a class to compute the F-statistic between two linear regression models.

3) Use it in the Kaggle playground competition.

3.3) SUBSET SELECTION

- Prediction accuracy
- Interpretation

Objective: Select a subset of $\hat{\beta} = (\beta_0, \beta_1, \dots, \beta_p)$

3.3.1) BEST-SUBSET SELECTION

- Find the subset of size k ($k \in \{0, 1, \dots, p\}$) that gives the smallest residual sum of squares:

$$RSS = \sum_{i=1}^n (y_i - X\beta)^2 \quad \begin{matrix} \dim(X) = n \times k \\ \dim(\beta) = k \end{matrix}$$

- Computationally expensive. Why?

How many ways are there of selecting k elements from p elements?

$$\binom{p}{k} = \frac{p!}{k!(p-k)!} \quad n! = n \cdot (n-1) \cdot \dots \cdot 2 \cdot 1$$

→ 'p choose k'

3.3.2) FORWARD- AND BACKWARD-STEPWISE SELECTION

- Sequentially add into the model the predictor that most improves the fit. (Forward)
- Sequentially subtract from the model the predictor that most preserves the fit. (Backward). Works by dropping the lowest z-score.

* Computationally more efficient, greedy algor: lhm, usually similar performance to best subset.

3.4) SHRINKAGE METHODS

Continuous version of subset selection.

3.4.1) RIDGE REGRESSION

- Shrinks coefficients by imposing a penalty on their size.

$$\hat{\beta}_R = \underset{\beta}{\operatorname{argmin}} \left(\sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right)$$

$$\beta = (\beta_1, \beta_2, \dots, \beta_p) - \text{Exclude } \beta_0$$

λ : Shrinkage parameter

Equivalently,

$$\hat{\beta}_R = \underset{\beta}{\operatorname{argmin}} \left(\sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 \right), \quad \sum_{j=1}^p \beta_j^2 \leq t$$

*) There is a 1-1 correspondence between t & λ .

We normalise the inputs. Can you tell why? How do we exclude β_0 ? Two ways:

1) Remove the column of 1s from X and do:

$$x_j = \frac{X_{ij} - \mu_{x_j}}{\sigma_{x_j}}, \quad y = y - \mu_y$$

$$\begin{aligned} \mu_{x_j} &= \frac{1}{N} \sum_{i=1}^N x_{ij}, & \sigma_{x_j}^2 &= \frac{1}{N} \sum_{i=1}^N (x_{ij} - \mu_{x_j})^2 \\ \mu_y &= \frac{1}{N} \sum_{i=1}^N y_i \end{aligned}$$

$$\text{Then } \hat{\beta} = (X^T X + \lambda I)^{-1} X^T y \quad (I \text{ has dim } p \times p)$$

$$\hat{\beta}_0 = \mu_y - \mu_x^T \hat{\beta} \quad (\mu_x \text{ has dim } p)$$

2) Change I for $D = \begin{bmatrix} 0 & 0 & \dots \\ 0 & 1 & 0 \\ \vdots & 0 & 1 \end{bmatrix}$, then

$$\hat{\beta} = (X^T X + \lambda D)^{-1} X^T y \rightarrow \beta_0 \text{ included}$$

3.4.2) THE LASSO

$$\hat{\beta}_L = \underset{\beta}{\operatorname{argmin}} \left[\sum_{i=1}^N (y_i - \beta_0 - \sum_{j=1}^p x_{ij} \beta_j)^2 + \lambda \sum_{j=1}^p |\beta_j| \right]$$

- No analytical solution, gradient descent?

Exercises (Certificate of engagement)

1) Improve the LinearRegression class from Session 1 to include:

- Best subset selection
- Forward & BACKWARD-Stepwise selection
- Ridge regression

2) Try your class methods on the three datasets:

- What results do you obtain
- Which one works best? Why?

3) Implement the solution for the Lasso using gradient descent (Hard).

REGRESSION BY SUCCESSIVE ORTHOGONALISATION

RECAP

Linear regression: $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p + \epsilon$

In matrix form $Y = XB + \epsilon$

We estimate $\beta = [\beta_0 \ \beta_1 \ \dots \ \beta_p]^T$ by $\hat{\beta} = [\hat{\beta}_0 \ \hat{\beta}_1 \ \dots \ \hat{\beta}_p]$

minimising $RSS(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (+ \lambda \sum_{j=1}^p \beta_j^2)$

Setting $\frac{\partial RSS}{\partial \beta} = 0 \Rightarrow \hat{\beta} = (X^T X)^{-1} X^T Y$

$\hat{\beta} = (X^T X + \lambda I)^{-1} X^T Y$ (Ridge)

Regression by successive orthogonalisation

Model: $y_i = \beta_0 + \sum_{j=1}^p \beta_j x_{ij} + \epsilon_i$

Goal: compute $\hat{\beta} = [\beta_0 \ \beta_1 \ \dots \ \beta_p]^T$

Step 1 | For each predictor

compute: $\bar{x}_j = \frac{1}{n} \sum_{i=1}^n x_{ij}$ and $\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$

and we center the data. For every observation:

$\tilde{x}_{ij} = x_{ij} - \bar{x}_j$ { Similar to
 $\tilde{y}_i = y_i - \bar{y}$ { normalising

No intercept!
Can you see why?

The regression task becomes $\tilde{y}_i = \sum_{j=1}^p \beta_j \tilde{x}_{ij} + \epsilon$

Step 2 | Core idea

Two predictors are orthogonal if $x_j^T x_k = 0$ ($k \neq j$).

All predictors are orthogonal if $x_j^T x_k = 0 \ \forall k, j$ with $k \neq j$

If that is the case, then $\beta_j = \frac{x_j^T y}{x_j^T x_j}$

Idea: build new orthogonal vectors $z_1, \dots, z_p$

Step 3 | Core algorithm (Gram-Schmidt)

1) Set $z_1 = \tilde{x}_1$

2) Compute z_2 :

Remove from $\tilde{x}_2$ the part explained by z_1 .

$$a_{12} = \frac{z_1^T \tilde{x}_2}{z_1^T z_1}, \quad z_2 = \tilde{x}_2 - a_{12} z_1$$

You can check $z_1^T z_2 = 0 \rightarrow$ Orthogonal

3) For z_3 (and z_j)

$$z_3 = \tilde{x}_3 - a_{13} z_1 - a_{23} z_2, \quad a_{kj} = \frac{z_k^T \tilde{x}_j}{z_k^T z_k}$$

Generally,

$$z_j = \tilde{x}_j - a_{1j} z_1 - a_{2j} z_2 - \dots - a_{(j-1)j} z_{(j-1)}$$

Pseudocode:

For j in $1 \dots p$:

$z_j = \tilde{x}_j$

for k in $1 \dots j-1$:

$$a_{kj} = \text{dot}(z_k, \tilde{x}_j) / \text{dot}(z_k, z_k)$$

$$z_j = z_j - a_{kj} z_k$$

Step 4 | Now:

$$\tilde{y} = \sum_{j=1}^p \hat{\gamma}_j z_j \quad \hat{\gamma}_j = \frac{z_j^T \tilde{y}}{z_j^T z_j} \quad (\hat{\beta} \neq \hat{\gamma})$$

We can get predictions, but how do we obtain the original $\hat{\beta}$?

Step 5 | In matrix form, we have:

$$\tilde{X} = ZR \quad R = \begin{bmatrix} 1 & a_{12} & a_{13} & \dots \\ 0 & 1 & a_{23} & \dots \\ 0 & 0 & 1 & \dots \\ \vdots & \vdots & \vdots & \ddots \end{bmatrix}$$

$$\text{and } y = R\beta \Rightarrow \beta = R^{-1} \gamma$$

Back substitution: $\rightarrow$ Since R is upper triangular

$$\hat{\beta}_p = \gamma_p \quad (\text{Start with last predictor})$$

$$\hat{\beta}_j = \gamma_j - \sum_{k=j+1}^p R_{jk} \hat{\beta}_k$$

$$\hat{\beta}_0 = \bar{y} - \sum_{j=1}^p \hat{\beta}_j \bar{x}_j$$

□

Why bother?

- Numerically stable, faster, standard in numerical libraries.
- Statistical interpretation of OLS

Exercises

1) Implement functions to center a vector and center the columns of a matrix.

2) Implement Gram-Schmidt

Input: X
Output: Z, R { Verify $X \approx Z @ R$

3) Compute the $\hat{\gamma}$ coefficients

4) Obtain $\hat{\beta}$

5) Create a full OLS via orthogonalisation function and compare your results with $\hat{\beta} = (X^T X)^{-1} X^T Y$ (Vanilla OLS)

MORE ON LINEAR REGRESSION & APPLICATIONS

LEAST ANGLE REGRESSION (LAR)

- Extremely efficient algorithm for computing the Lasso path
- At each step we identify the variable that is most correlated with the target. Then, we move it towards its OLS coefficient.

Algorithm

- 1) Normalise predictors. Set $r = y - \bar{y}$, $\beta_1, \beta_2, \dots, \beta_p = 0$
- 2) Find x_j most correlated with r
- 3) Move β_j towards its OLS coefficient until some other x_k has as much correlation with r
- 4) Move β_j, β_k towards OLS until some other x_l has as much correlation with r (now x_l/β_l is in the active set)
- 5) Continue this way until all p predictors have been entered $\min(N-1, p)$ steps

* Let A_k be the active set. At the beginning of the k -th step. Let β_{A_k} be the coefficient vector at that step. Then:

$$r_k = y - X_{A_k} \beta_{A_k} \quad (\text{residual})$$

$$s_k = (X_{A_k}^T X_{A_k})^{-1} X_{A_k}^T r_k \quad (\text{direction of the current step})$$

$$\beta_{A_k}(\alpha) = \beta_{A_k} + \alpha s_k \quad (\alpha: \text{step size})$$

$$c = X^T r \quad (\text{Correlations}) \quad -1 \leq \cos(\alpha) \leq 1$$

$$c_j = x_j^T r = \|x_j\| \|r\| \cos \theta_j = \|r\| \cos \theta_j$$

$\Rightarrow c_j = \|r\| \cos \theta_j$ (Picking highest $|c_j| \Leftrightarrow$ Picking small θ_j)

LEAST ANGLE REGRESSION (Lasso Modification)

Algorithm: If a non-zero coefficient hits zero, drop it from the active set and recompute current joint least squares.

LINEAR REGRESSION IN PRACTICE (CAPM)

$$r_i = \alpha_i + \beta_i r_m + \varepsilon_i$$

r_i : stock/strategy returns
 r_m : market returns

$$\varepsilon_i: \text{idiosyncratic noise} \quad \alpha_i: \text{systematic mispricing}$$

$$\hat{\varepsilon}_i = r_i - \hat{\alpha}_i - \hat{\beta}_i r_m \quad (\text{residuals})$$

$$\text{Recall: } \hat{\theta} = \begin{bmatrix} \hat{\alpha}_i \\ \hat{\beta}_i \end{bmatrix} = (X^T X)^{-1} X^T r_i \quad (\text{OLS})$$

$$\text{Var}(\hat{\theta}) = \sigma^2 (X^T X)^{-1} \quad (\widehat{\text{Var}}(\hat{\theta}) = \hat{\sigma}^2 (X^T X)^{-1})$$

$$\hat{\sigma}^2 = \frac{1}{N-2} \hat{\varepsilon}^T \hat{\varepsilon}, \quad \hat{\varepsilon} = y - X \hat{\theta}$$

$$1-\gamma \text{ confidence interval: } \hat{\alpha} \pm z_{1-\gamma/2} \cdot \text{SE}(\hat{\alpha}) \quad (\text{SE}(\hat{\alpha}) = \sqrt{\text{Var}(\hat{\alpha})})$$

$$95\% \text{ CI: } z_{1-\gamma/2} = 1.96\%$$

$$\text{Assuming } \text{Cov}(r_m, \varepsilon_i) = 0 \quad \text{Var}(A+B) = \text{Var}(A) + \text{Var}(B) + 2\text{Cov}(A, B)$$

$$\text{Var}(r_i) = \text{Var}(\alpha_i + \beta_i r_m + \varepsilon_i) \\ = \text{Var}(\alpha_i) + \text{Var}(\beta_i r_m + \varepsilon_i) + 2\text{Cov}(\alpha_i, \beta_i r_m + \varepsilon_i)$$

$$\text{Cov}(X, Y) = E[(X - E[X])(Y - E[Y])] \quad (\alpha_i \text{ is treated as a const.})$$

$$E[\alpha_i] = \alpha_i \Rightarrow \text{Cov}(\alpha_i, \beta_i r_m + \varepsilon_i) = 0 \quad \text{Also } \text{Var}(\alpha_i) = 0$$

$$\Rightarrow \text{Var}(r_i) = \text{Var}(\beta_i r_m + \varepsilon_i) \\ = \text{Var}(\beta_i r_m) + \text{Var}(\varepsilon_i) + 2\text{Cov}(\beta_i r_m, \varepsilon_i) \\ = \beta_i^2 \text{Var}(r_m) + \sigma_{\varepsilon_i}^2 \quad (\text{Var}(cX) = c^2 \text{Var}(X)) \\ = \text{systematic risk} + \text{idiosyncratic risk}$$

For your portfolio

$$r_p = w^T \alpha + (w^T \beta) r_m + w^T \varepsilon$$

$$\text{Ideally } w^T \beta = 0 \Rightarrow r_p = w^T \alpha + w^T \varepsilon$$

$$\text{Objective: } \max_w w^T \alpha \quad \text{subject to } w^T \beta = 0$$

Example for two assets:

$$\max_w w_1 \alpha_1 + w_2 \alpha_2 \quad \text{with } w_1 \beta_1 + w_2 \beta_2 = 0$$

$$\Rightarrow \frac{w_1}{w_2} = -\frac{\beta_2}{\beta_1} \\ \Rightarrow \begin{cases} w_1 = \lambda \beta_2 \\ w_2 = -\lambda \beta_1 \end{cases} \quad \lambda \in \mathbb{R}$$

$$\therefore w^T \alpha = \lambda(\beta_2 \alpha_1 - \beta_1 \alpha_2)$$

$$\beta_2 \alpha_1 - \beta_1 \alpha_2 > 0 \Rightarrow \lambda > 0$$

$$\beta_2 \alpha_1 - \beta_1 \alpha_2 < 0 \Rightarrow \lambda < 0$$

Exercises

- 1) Compute $\hat{\alpha}, \hat{\beta}$ for a stock of your choice
- 2) Estimate $\text{SE}(\hat{\alpha}), \text{SE}(\hat{\beta})$ and a 95% CI
- 3) Create a β neutral portfolio ($w^T \beta = 0$) by trading the market
- 4) Repeat for other stocks, try and combine 2 or more stocks in your portfolio
- 5) Create a function to compute the Lasso using the LAR algorithm (Hard)